

A line is a dot that went for a walk.
Paul Klee

In Chapter 4, we delved into how the mathematical models emphasized in this book adhere to linearity. This necessity extends to both the objective function and every constraint, stipulating that decision variables can only appear in linear combinations. The requirement seems quite stringent at first glance and indeed prevents some problems from being tackled and solved with the solution methods described in Chapters 6, 7, and 8. Notwithstanding, **many non-linear operators can be mathematically linearized so that the resulting model is linear**. The price to pay, because of the “no free lunch” theorem¹, is that **the linearization process generally entails the addition of auxiliary decision variables and/or constraints, hence contributing to the increase of the complexity of the original mathematical model**.

Some readers might be surprised to realize that, in Chapter 4, we already “implicitly” applied some linearization techniques. One example is the either-or constraint from Section 4.8.2 because such a constraint maps the (non-linear) logical operator $\vee^2$. The “algorithmic” price we pay is the addition of binary decision variable y . Another fitting example is the fixed charge constraint (Section 4.8.4). If we assume that the initial fixed charge cost C allows the generation of revenue proportionally to x , where x maps the level of productivity or sales (e.g., aircraft sales in Example 4.7), via parameter R (revenue per unit x), we can define the profit P as

$$P = \begin{cases} 0 & \text{if } x=0 \\ Rx - C & \text{otherwise} \end{cases} \tag{5.1}$$

which is a non-linear function featuring a discontinuity in $x = 0$. Leveraging the addition of binary variable y both in the constraints and the objective facilitates the linearization of such a discontinuity.

Despite their inherent non-linear nature, we opted to incorporate the aforementioned constraints in Chapter 4 because they are “standard” constraint types as per OR standards. In the subsequent sections, we shift our focus to other constraint types. Although widely utilized in mathematical modeling, these types are more renowned (or notorious, depending on the context) for their non-linear characteristics, necessitating a linearization process.

5.1 Product of decision variables

Often, our problem formulations necessitate representing quantities of interest through the product of two decision variables. For instance,

5.1 Product of decision variables 45

5.1.1 Product of two binary variables 46

5.1.2 Product of a binary and a continuous decision variable 48

5.2 Absolute value 49

5.3 Piecewise linear formulations 51

5.4 If-else statement 53

1: For a more formal definition of such a theorem, we refer interested readers to this [Wikipedia page](#).

2: Logical or. See this [Wikipedia page](#) for more info.

consider a scenario involving the Champions League final. If two specific teams (indexed i and j) qualify for the final, we aim to capture this event in our mathematical model. We could define binary variables x_i and x_j , each taking a value of 1 if team i or team j reaches the final, respectively. Consequently, we can introduce another binary variable $y_{ij} = x_i x_j$, which equals 1 if both teams qualify for the final ($y_{ij} = 1 \times 1 = 1$), and 0 otherwise. However, this relationship is non-linear due to the product of two binary decision variables. Fortunately, it is linearizable, as described in Section 5.1.1. Similarly, situations may arise where we encounter the product of a binary and a continuous decision variable, which is also linearizable, as discussed in Section 5.1.2.

5.1.1 Product of two binary variables

We already introduced, for this specific case, our intended goal. Given binary variables x_i and x_j , our goal is to capture $y_{ij} = x_i x_j$, but in a linear fashion. We achieve this by defining the following constraints:

$$y_{ij} \geq x_i + x_j - 1 \quad (5.2)$$

$$y_{ij} \leq x_i \quad (5.3)$$

$$y_{ij} \leq x_j \quad (5.4)$$

$$y_{ij}, x_i, x_j \in \{0, 1\} \quad (5.5)$$

where (5.2) is the key constraint. It forces y_{ij} to be unitary if both x_i and x_j are unitary $y_{ij} \geq 1 + 1 - 1 = 1 \rightarrow y_{ij} = 1$ and leaves it the freedom to be either 1 or 0 otherwise. If all the y_{ij} decision variables appear in the objective function in a term that should be minimized, then the model will assign them a value of 0 as this is beneficial for the objective. In such a case, (5.3)-(5.4) are not strictly needed, but they help with the linear relaxation of the problem (see Chapters 6-8). They force y_{ij} to be 0 as soon as one of the two original binary variables is 0 ($x_i = 0 \rightarrow y_{ij} \leq 0 \rightarrow y_{ij} = 0$. The same applies if $x_j = 0$). Similarly, in the case of a maximization problem, the model will favor assigning y variables to be 1 all the time. Yet having (5.3)-(5.4) prevents this as soon as one variable between x_i and x_j is not unitary. We showcase an application of such a linearization in Example 5.1.

Example 5.1 *In many hub airports worldwide, a significant portion of passengers are transfer passengers, meaning the hub airport neither marks the origin nor the destination of their journey. For such airports, a critical KPI is the connection time, closely linked to the distance for transfer passengers. Requiring transfer passengers to disembark from their first flight at one end of the airport and then traverse a considerable distance to connect to their subsequent flight can lead to discomfort and impede the overall passenger experience. The problem of efficiently assigning aircraft to gates is generally called the Gate Assignment Problem (GAP), and can have different objectives reflecting the different needs of the main stakeholders involved, namely the airport, the airlines, and passengers. In this example, we employ the perspective of the passengers. The airport knows the set of inbound/outbound flights $\mathcal{F}$ (indexed by f) that are to be operated on a specific day. The airport is also characterized by a set of gates $\mathcal{G}$ (indexed by g), such as the ones shown in Figure 5.1, where $D_{g_1 g_2}$ is the walking distance*

between gates g_1 and g_2 . Additionally, the airport has received by all airlines data on the number of transfer passengers $P_{f_1 f_2}$ that are expected to transfer inside the terminal from flight f_1 to flight f_2 . For example, if f_1 is a flight from Milan Linate Airport to Amsterdam Schiphol Airport, f_2 is a flight from Amsterdam Schiphol Airport to John F. Kennedy International Airport, and $P_{f_1 f_2} = 15$, the hub airport (Amsterdam Schiphol Airport in this case) is aware that 15 passengers are connecting there from Italy on their way to the United States. We aim to formulate a suitable objective function for the GAP, aiding the hub airport in minimizing the total distance traveled by transfer passengers within a given day.

The airport knows the overall number of transfer passengers expected during the analyzed day, as the total number is $\sum_{f_1 \in \mathcal{F}} \sum_{f_2 \in \mathcal{F}} P_{f_1 f_2}$. We consider each pair of flights and aggregate the number of transfer passengers between them. It is important to note that we should account for both directions, i.e., (f_1, f_2) and (f_2, f_1) . For instance, in the scenario mentioned earlier, passengers may also transfer in the Netherlands from the United States on their way to Italy. The crucial decision that drives our objective is how to assign aircraft to gates. In fact, if we assign flight f_1 to gate g_1 and flight f_2 to gate g_2 , the overall walking distance related to those two flights is $D_{g_1 g_2} (P_{f_1 f_2} + P_{f_2 f_1})$, which is obtained by multiplying the distance between the two gates by the number of transfer passengers in both directions (from f_1 to f_2 and vice versa).

Because the airport needs to assign flights to gates, a reasonable decision variable could be $x_{fg} \in \{0, 1\}$, taking unitary value if flight f is assigned to gate g . Then, our goal is to add to our objective function every term in the $D_{g_1 g_2} (P_{f_1 f_2} + P_{f_2 f_1})$ form if flight f_1 is assigned to gate g_1 and flight f_2 to gate g_2 . We could rewrite this statement in more mathematical terms as “if $x_{f_1 g_1} x_{f_2 g_2} = 1$ ” and notice the expression entails the product of two binary decision variables. Hence, we could rely on (5.2)-(5.4), with the complication that now each binary variable depends on two indices, and hence y depends on four indices. We define $y_{f_1 g_1 f_2 g_2} \in \{0, 1\}$ which takes a unitary value if flight f_1 is assigned to gate g_1 and flight f_2 to gate g_2 . If we sort flights by increasing order, i.e., $\mathcal{F} = \{1, 2, 3, \dots\}$, we should define such a decision variable $\forall f_1, f_2 \in \mathcal{F} : f_2 > f_1, g_1, g_2 \in \mathcal{G}$. **With $f_2 > f_1$ we mean that flight f_2 comes after flight f_1 in the set (basically, that index f_2 is greater than index f_1). This is done to reduce the variables to be defined. In fact, stating that f_1 is assigned to g_1 and f_2 to g_2 is equivalent to stating that f_2 is assigned to g_2 and f_1 to g_1 . Conversely, we need to cycle over all gates twice. As a matter of fact, $y_{f_1 g_1 f_2 g_2}$ is a valid decision variable as it allows for the assignment of two flights to the same gate g if they are sufficiently spaced apart in time.** We can fully formalize our y decision variables as:

$$y_{f_1 g_1 f_2 g_2} \geq x_{f_1 g_1} + x_{f_2 g_2} - 1 \quad \forall f_1, f_2 \in \mathcal{F} : f_2 > f_1, g_1, g_2 \in \mathcal{G} \quad (5.6)$$

$$y_{f_1 g_1 f_2 g_2} \leq x_{f_1 g_1} \quad \forall f_1, f_2 \in \mathcal{F} : f_2 > f_1, g_1, g_2 \in \mathcal{G} \quad (5.7)$$

$$y_{f_1 g_1 f_2 g_2} \leq x_{f_2 g_2} \quad \forall f_1, f_2 \in \mathcal{F} : f_2 > f_1, g_1, g_2 \in \mathcal{G} \quad (5.8)$$

Thanks to the introduction of the $y_{f_1 g_1 f_2 g_2}$ decision variables, we can now express our objective function (in a linear fashion) as:



Figure 5.1: Aircraft gated at Amsterdam Schiphol Airport.

$$\min \sum_{f_1 \in \mathcal{F}} \sum_{f_2 \in \mathcal{F}: f_2 > f_1} \sum_{g_1 \in \mathcal{G}} \sum_{g_2 \in \mathcal{G}} D_{g_1 g_2} (P_{f_1 f_2} + P_{f_2 f_1}) y_{f_1 f_2 g_1 g_2} \quad (5.9)$$

(5.9) expresses the overall walking distance of transfer passengers and depends on the gate assignment choices that the airport performs. Note that we purposely decided not to provide a full GAP formulation, as there is no standard formulation in the first place. Apart from some generic constraints (e.g., every flight f should be assigned to a gate g : $\sum_{g \in \mathcal{G}} x_{fg} = 1 \forall f \in \mathcal{F}$), there is a lot of variability related to the type of airport, the modeling assumptions, the stakeholder's perspective when deciding the objective function. We refer interested readers to Daş et al., 2020 for a comprehensive review of GAP formulations. Finally, we comment on the size of the decision variable set y for large hubs in **⚡ A note on the number of $y_{f_1 f_2 g_1 g_2}$ decision variables in the GAP box.**

⚡ A note on the number of $y_{f_1 f_2 g_1 g_2}$ decision variables in the GAP

Hub airports such as Amsterdam Schiphol Airport can handle more than 1,000 flights per day and can be characterized by roughly 200 gates. It should be noted that a flight can generally be assigned only to a subset of gates because of customs restrictions (e.g., Schenghen vs. non-Schenghen flights), airline preferences, or other reasons. Notwithstanding, let us assume a hub airport that, on a given day, must handle $|\mathcal{F}| = 1,000$ flights and that each flight can be assigned to a subset $\mathcal{G}_f$ of gates that “only” comprises 10 options. Hence, we could get a rough estimate of the number of $y_{f_1 f_2 g_1 g_2}$ decision variables needed as:

$$\begin{aligned} |y_{f_1 f_2 g_1 g_2}| &= \frac{|\mathcal{F}|(|\mathcal{F}| - 1)}{2} \times \mathcal{G}_f \times \mathcal{G}_f \\ &= \frac{1,000 \times 999}{2} \times 10 \times 10 \simeq 50,000,000 \end{aligned}$$

which highlights the complexity of such a problem for large hub airports. **It is also important to highlight that the GAP is generally not solved for a full day, but smaller planning windows are solved sequentially (which entails more manageable problem sizes). Because of flight delays and many other unforeseen circumstances, a GAP solution will be subject to continuous changes throughout the day anyway. We have all experienced at least once in our lifetime a sudden gate change for one of our flights!**

5.1.2 Product of a binary and a continuous decision variable

3: Note: the decision variable can also be an integer with no changes in the linearization process.

Some mathematical formulations might require the product of a continuous³ and a binary variable in the constraints or the objective. Let $x \in \{0, 1\}$ be the binary variable and $y \leq U$ the continuous variable where U is a maximum value (upper bound) such a variable can take. Introducing a new continuous variable z , defining $z = xy$ achieves

our intended goal in a non-linear fashion. The linearization of this relation entails the reworking of the xy product via the following set of constraints:

$$z \leq Ux \quad (5.10)$$

$$z \leq y \quad (5.11)$$

$$z \geq y - U(1 - x) \quad (5.12)$$

$$x \in \{0, 1\} \quad (5.13)$$

$$y \leq U \quad (5.14)$$

$$z \geq 0 \quad (5.15)$$

where in (5.10) and (5.12) U plays the role of a big- M .

Let us verify that the intended goal is achieved by analyzing the behavior of (5.10)-(5.12) for different combinations of x and y . If $x = 0$ and $y = 0$ we obtain $z \leq 0$, $z \leq 0$, and $z \geq 0$, which implies $z = 0$. If $x = 0$ and $y > 0$ we obtain $z \leq 0$, $z \leq y$, and $z \geq y - U$, which implies $z = 0$. If $x = 1$ and $y = 0$ we obtain $z \leq U$, $z \leq 0$, and $z \geq 0$, which implies $z = 0$. Finally, if $x = 1$ and $y > 0$ we obtain $z \leq U$, $z \leq y$, and $z \geq y$, which implies $z = y$ as requested.

5.2 Absolute value

Some mathematical formulations rely on the absolute value of the difference between two continuous or integer variables to function correctly. Consider a model tasked with efficiently packing two-dimensional boxes into a two-dimensional bin along an $x - z$ vertical plane. Due to Earth's gravity, floating boxes are not allowed. Therefore, box i can only have a z -coordinate greater than zero if stacked on top of another box j . This requirement can be verbalized as *"if the upper side of box j has the same height as the lower side of box i , then box i can be stacked on top of box j "*.

We can mathematically translate the absolute value $|x - y|$ of the difference of continuous decision variables x and y with the following set of equations:

$$x - y \leq z_{xy} \quad (5.16)$$

$$y - x \leq z_{xy} \quad (5.17)$$

$$z_{xy} \leq x - y + M(1 - m_{xy}) \quad (5.18)$$

$$z_{xy} \leq y - x + Mm_{xy} \quad (5.19)$$

$$m_{xy} \in \{0, 1\} \quad (5.20)$$

$$x, y, z_{xy} \geq 0 \quad (5.21)$$

where z_{xy} is an auxiliary continuous variable which is ensured to be equal to $|x - y|$, M is an upper bound (big- M) on the value z_{xy} can take, and $m_{xy} \in \{0, 1\}$ takes a unitary value if $x \geq y$. Let us verify the

validity of (5.16)-(5.19) considering the three distinct cases $x - y = \alpha > 0$, $x - y = 0$, and $x - y = \alpha < 0$.

In the first case ($x - y = \alpha > 0$), we have

$$\begin{aligned}\alpha &\leq z_{xy} \\ -\alpha &\leq z_{xy} \\ z_{xy} &\leq \alpha \\ z_{xy} &\leq \alpha + M\end{aligned}$$

4: Note; we define *dummy* a constraint that is always satisfied. Dummy constraints are found oftentimes in formulations where big- M s and a binary variable appear in two linked constraints. Depending on the value taken by the binary, one of the two constraints will be active and the other dummy (or vice versa). Previously in the book, we also used the term de-activated. Hence, we might be using the terms dummy, redundant, or de-activated interchangeably.

with the second and fourth constraints being dummy⁴ constraints, i.e., redundant. The first and third constraints ($z_{xy} \geq \alpha$ and $z_{xy} \leq \alpha$) imply $z_{xy} = |x - y| = x - y$ as required.

In the second case ($x = y$), we have:

$$\begin{aligned}0 &\leq z_{xy} \\ 0 &\leq z_{xy} \\ z_{xy} &\leq 0 \\ z_{xy} &\leq M\end{aligned}$$

with the fourth constraint being dummy. The first (or second) and third constraints ($z_{xy} \geq 0$ and $z_{xy} \leq 0$) imply $z_{xy} = |x - y| = 0$ as required.

Note that this second case is the limit case of both the first and third one when $\alpha = 0$, but we opted to highlight it as a special case of its own.

Finally, in the third case ($y - x = \alpha > 0$), we have

$$\begin{aligned}-\alpha &\leq z_{xy} \\ \alpha &\leq z_{xy} \\ z_{xy} &\leq -\alpha + M \\ z_{xy} &\leq \alpha\end{aligned}$$

with the first and third constraints being dummy. The second and fourth constraints ($z_{xy} \geq 0$ and $z_{xy} \leq 0$) imply $z_{xy} = |x - y| = y - x$ as required.

Note that, in some formulations, it may be much easier to handle absolute value cases. For example, if we have a constraint:

$$|x_1 - x_2| \leq 2$$

it is easy to represent the same with two constraints as follows:

$$x_1 - x_2 \leq 2 \tag{5.22}$$

$$-x_1 + x_2 \leq 2 \tag{5.23}$$

which actually represents the feasible region given in Figure 5.2.

However, we need to be careful if we have the case with

$$|x_1 - x_2| \geq 2$$

which creates a non-convex region as given in Figure 5.3 where the same trick cannot be used. In this case, we have two disjoint feasible regions and we can resort to the techniques discussed in Section 4.8.2.

5.3 Piecewise linear formulations

In some mathematical formulations, the objective function and/or constraints may be represented by a nonlinear function such as polynomials or an exponential curve. For example, the profit as the objective function of a mathematical model might have a diminishing rate of returns depending on decision variable x as in Figure 5.4. This logarithmic function can be approximated by a piecewise linear curve as already indicated on the figure. Therefore, the profit curve can be represented by those three linear segments with the variable profits of c_1 , c_2 , and c_3 respectively, corresponding to their slopes.

In order to represent this piecewise linear relation, we need to adapt the mathematical formulation. First of all, we need to represent the original decision variable x in terms of three new decision variables corresponding to the linear segments as follows:

$$x = \delta_1 + \delta_2 + \delta_3 \quad (5.24)$$

where δ_1 is the linear segment for the values of x between 0 and the first break point b_1 , δ_2 corresponds to the segment between b_1 and b_2 and finally δ_3 corresponds to the segment between b_2 and b_3 . The δ variables are therefore given as follows:

$$0 \leq \delta_1 \leq b_1 \quad (5.25)$$

$$0 \leq \delta_2 \leq b_2 - b_1 \quad (5.26)$$

$$0 \leq \delta_3 \leq b_3 - b_2 \quad (5.27)$$

The objective function then can be reformulated as:

$$\max \quad c_1\delta_1 + c_2\delta_2 + c_3\delta_3 \quad (5.28)$$

so that it approximates the original full line representing the non-linear function $f(x)$ in Figure 5.4 with the piecewise linear dashed line. **It follows that the more segments we choose, the more accurate (in general) the approximation, at the cost of an increased size of the linearized model.**

For this piecewise linear transformation to be valid, we need to ensure that $\delta_1 = b_1$ whenever $\delta_2 > 0$ and similarly that $\delta_2 = b_2 - b_1$ whenever $\delta_3 > 0$.

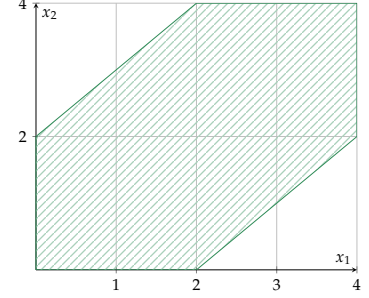


Figure 5.2: Feasible region with the absolute value constraint - convex case.

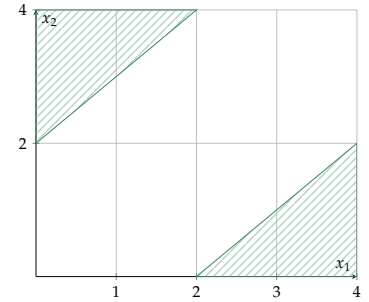


Figure 5.3: Feasible region with the absolute value constraint - non-convex case.

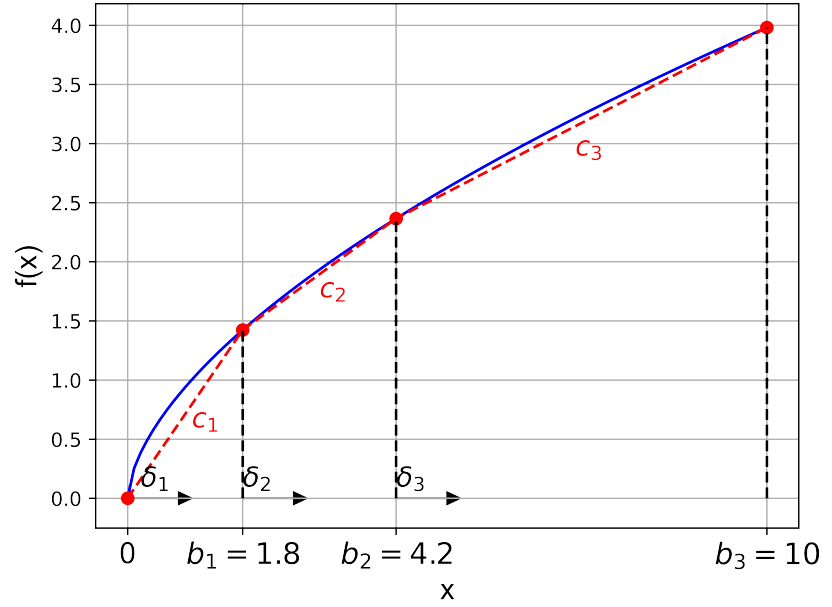


Figure 5.4: Piecewise linear approximation of a non-linear function.

As per (5.28), we are reconstructing x as the summation of the three segments, but we only need the second one if we exceed the bound b_1 of the first one and the third one if we exceed the bound b_2 of the second segment. Note that if $c_1 > c_2 > c_3$ this will already be ensured in case of a maximization problem as we have at hand. Nevertheless, to have a general formulation that addresses other cases we need to define binary variables to ensure this. Let us define:

$$\begin{aligned} w_1 &\in \{0, 1\}, 1 \text{ if } \delta_1 = b_1, 0 \text{ otherwise} \\ w_2 &\in \{0, 1\}, 1 \text{ if } \delta_2 = b_2, 0 \text{ otherwise.} \end{aligned}$$

Namely, if $w_1 = w_2 = 0$, only the first line segment is active and $x \leq b_1$. If $w_1 = 1$ and $w_2 = 0$, the first and second line segments are active and $b_1 \leq x \leq b_2$. Finally, if $w_1 = w_2 = 1$, all three segments are active and $x \geq b_2$. **Therefore, we represent the decision variable x through three linear segments with the definition of three continuous variables (δ) and two binary decision variables (w). The number of the binary decision variables is always equal to the number of linear segments minus one.**

Considering the above definitions and requirements, we can provide the full formulation as follows:

$$\max \quad c_1\delta_1 + c_2\delta_2 + c_3\delta_3 \quad (5.29)$$

s.t.:

$$b_1w_1 \leq \delta_1 \leq b_1 \quad (5.30)$$

$$(b_2 - b_1)w_2 \leq \delta_2 \leq (b_2 - b_1)w_1 \quad (5.31)$$

$$0 \leq \delta_3 \leq (b_3 - b_2)w_2 \quad (5.32)$$

$$w_1, w_2 \in \{0, 1\} \quad (5.33)$$

This formulation can be applied to piecewise linear curves with any number of segments. In that case, we would need to have a general variable δ_j for each segment j with a length of L_j , and the constraints will read:

$$L_j w_j \leq \delta_j \leq L_j w_{j-1} \quad (5.34)$$

5.4 If-else statement

In some mathematical formulations, we might require a constraint to become active if a certain choice is being made and to be redundant otherwise. This requirement can be interpreted as **an if-else statement where if a certain condition is met, the constraint is active, while it is made redundant otherwise**. We show a standard application in Example 5.2.

Example 5.2 *Let us consider a truck that must perform deliveries to a set of customers $\mathcal{C}$ (indexed by i or j) scattered across town. We need to assist the trucking company in assessing the best sequence of customers to visit, given that they all have specified different preferred delivery times. Hence, we want to deliver everything on time while avoiding unnecessary detours.*

In Section 4.1, we already anticipated that a routing decision variable x_{ij} maps, if unitary, that the truck moves from customer i to customer j . In addition, because keeping track of time is important in this problem to avoid delays, we can define continuous decision variable t_i as the time when our truck starts the delivery service at customer $i \in \mathcal{C}$. Additionally, we assume that performing the delivery at a customer $i \in \mathcal{C}$ takes P_i time-units and that the traveling time between customers i and j requires T_{ij} time-units. In such a setting, keeping track of time is easy if a sequence of customers is pre-assigned to us. Let us assume the first three customers are, in sequence, $c = 1, 2, 3$ and that the truck starts the journey from a depot indexed by 0 at time 0. Hence, $t_1 = T_{01}$ implies that the start of service of customer 1 is simply equal to the traveling time from the depot to the customer (assuming we can start the service as soon as we arrive there). Conversely, $t_2 = t_1 + P_1 + T_{12} = T_{01} + P_1 + T_{12}$ implies that the start of service of customer 2 is equal to the start of service of customer 1 (t_1) plus the time needed for the delivery (P_1) and the traveling time between customers (T_{12}). Following the same logic, $t_3 = t_2 + P_2 + T_{23} = T_{01} + P_1 + T_{12} + P_2 + T_{23}$.

Our main issue is that we are not given a pre-assigned sequence of customers, as defining the most appropriate sequence is exactly our task. Hence, we need to enforce a **time-precedence constraint** such as

$$t_j \geq t_i + P_i + T_{ij} \quad (5.35)$$

only if the truck moves from i to j ⁵. This is exactly an if-else constraint type because we can rephrase the following requirement as *if the truck drives from i to j , then time-precedence constraint (5.35) must be enforced; else, it must be made redundant*. We visually display the situation in Figure 5.5, where we assume that the truck moves from i to j , but represent a third

5: Note that in (5.35) we do not have an equality, but an $\geq$ inequality. As in many applications the goal is to minimize time, the equality will hold. Another reason why the $\geq$ is preferred is that the truck might have to wait at the customer because it arrived too early. Hence the start of the service time could be delayed.

customer l symbolizing that the arc (l, j) is also an option we could consider in our mathematical model.

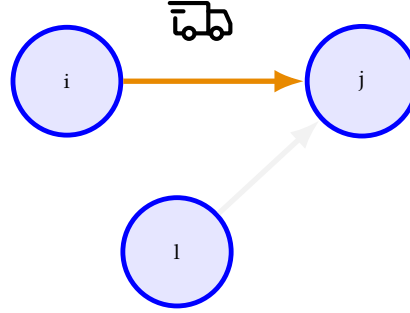


Figure 5.5: Representation of a customer pair (i, j) served in sequence by a truck. Because of the sequence, a time-precedence constraint $t_j \geq t_i + P_i + T_{ij}$ must hold.

We achieve the intended goal by leveraging the ability of a big- M constant to activate or de-activate a constraint depending on the value taken by a binary decision variable which, in this case, is x_{ij} . The correct form of the constraint is

$$t_j \geq t_i + P_i + T_{ij} - M_{ij}(1 - x_{ij}) \quad \forall i, j \in \mathcal{N} \quad (5.36)$$

where we are specifying that such a constraint must be enforced for every pair on nodes i, j in our network that comprises both the depot and all customers (with $\mathcal{N} = \{0, 1, \dots, |\mathcal{C}|\}$ we mean the full set of nodes). The additional term $M_{ij}(1 - x_{ij})$ satisfies our needs. If $x_{ij} = 1$, then we obtain the original constraint (5.35) that we want to enforce. If $x_{ij} = 0$, then we obtain

$$t_j \geq t_i + P_i + T_{ij} - M_{ij} \quad (5.37)$$

which can be re-written as

$$M_{ij} \geq t_i - t_j + P_i + T_{ij} \quad (5.38)$$

Note that in (5.36) (and as a consequence (5.37)-(5.38)) we use M_{ij} to reflect that the big- M can be customized for each (i, j) customer pair to avoid excessively large constants. Let us assume that every customer specifies a time-window where they would like their delivery to start being carried out, where the earliest time is E_i and the latest time is L_i . This constraint is stringent, implying that deliveries beyond the planned time-window are not permissible. We can extend this rigidity by defining analogous values for the depot, indicating the earliest time a truck can depart (start of shift) and the latest time it should return (end of shift). Therefore, in constraint (5.38), we can calculate the maximum possible value for the right-hand side as follows: since t_i is preceded by a positive sign, we substitute it with the latest (i.e., largest) permissible time node i can be visited, denoted as L_i . Conversely, as t_j is preceded by a negative sign, we substitute it with the earliest (i.e., smallest) value it can assume, denoted by E_j . Constants such as P_i and T_{ij} remain unchanged and should be preserved as they are. Hence, we could assign to every M_{ij} the value of

$$M_{ij} = \max \{0, L_i - E_j + P_i + T_{ij}\} \quad (5.39)$$

which guarantees that M_{ij} is “large enough” to make a constraint redundant, if needed, without being excessively large. Note that (5.39) displays a max operator with the first terms being 0. In fact, $L_i - E_j + P_i + T_{ij} < 0$ implies $E_j > L_i + P_i + T_{ij}$, meaning that the delivery at customer j will have to be carried out after the delivery at customer i (and not necessarily as the next one after i). **Hence, no M_{ij} is formally needed as $t_j \geq t_i + P_i + T_{ij}$ will be satisfied anyway.**

Referring back to Figure 5.5 and focusing on node j as the “destination” node, if we assume it is visited right after node i the application of (5.36) yields

$$t_j \geq t_i + P_i + T_{ij} \quad (5.40)$$

$$t_j \geq t_l + P_l + T_{lj} - \max \{0, L_l - E_j + P_l + T_{lj}\} \quad (5.41)$$

where (5.40) is active as the time-precedence between i and j must be enforced, while (5.41) is made redundant. Note that (5.41) does not imply that customer l is not visited before customer j , but that **they are not visited immediately before customer j , which is a substantial difference modeling-wise. The final routing could, for example, be $l \rightarrow i \rightarrow j$, and (5.40)-(5.41) would still hold.** We let readers verify that, in such a case, a similar time-precedence constraint is active between l and i as $x_{li} = 1$.

We conclude the section by providing a general form of the if-else constraint. Let us assume that our goal is to allow our model to activate or not a generic expression $\sum_{j \in x} A_{ij}x_j \leq b_i$ using a binary decision variable y_i . Then, the generic form of an if-else constraint can be expressed as

$$\sum_{j \in x} A_{ij}x_j - b_i \leq M(1 - y_i) \quad (5.42)$$

where in (5.42) the constraint is active if $y_i = 1$ and redundant if $y_i = 0$. In Part IV-Part V we will showcase several if-else constraints playing a crucial role in different mathematical models.